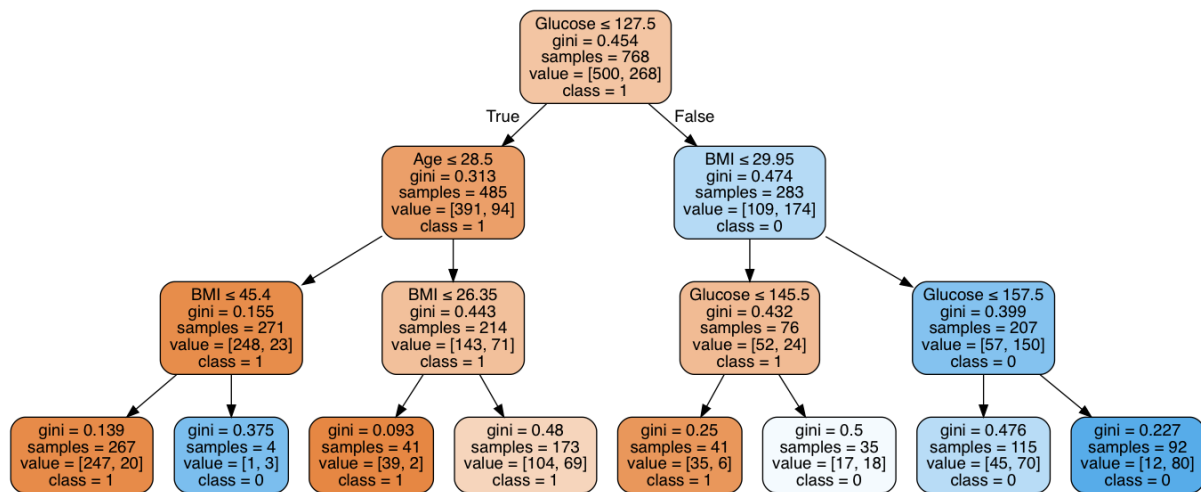
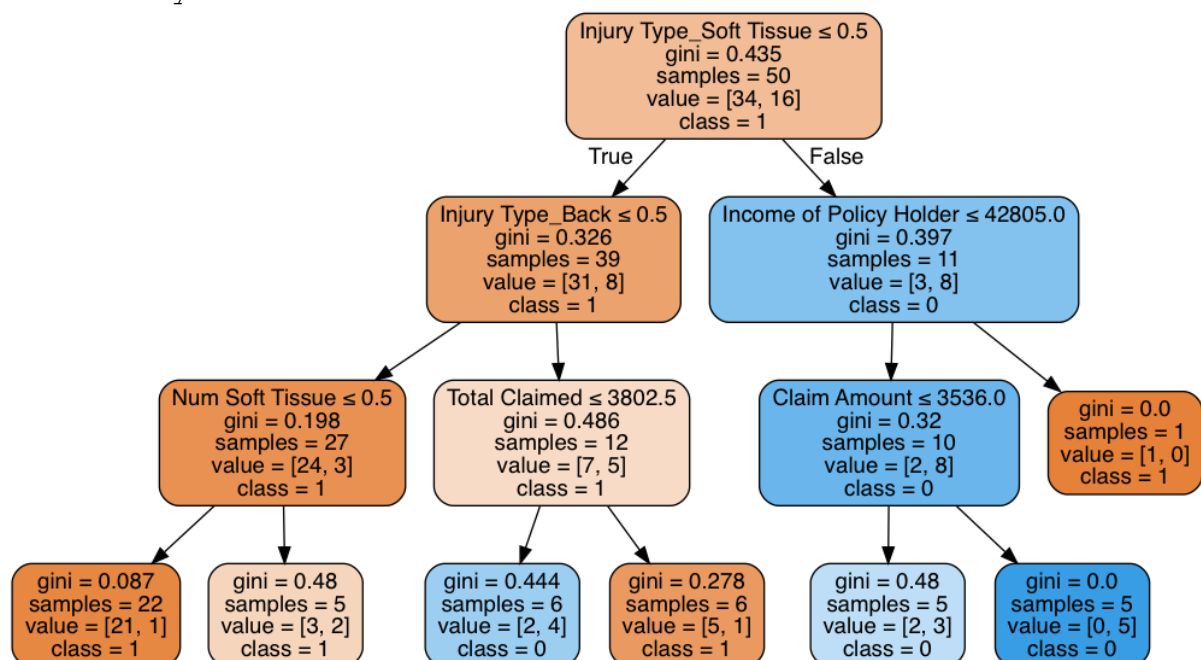


1) Run the example on the Indian diabetes dataset. Plot the decision tree for the gini measure with a max depth of 3.



2) Now apply the online example to data from class. Use the insurance fraud dataset we discussed in class. Split the data into a 10% train and 90% test set using `random_state = 1`. Create a decision tree with a max depth of 3 using a gini measure. Print the accuracy on the test set and the tree. Is this a good approach? Why or why not?

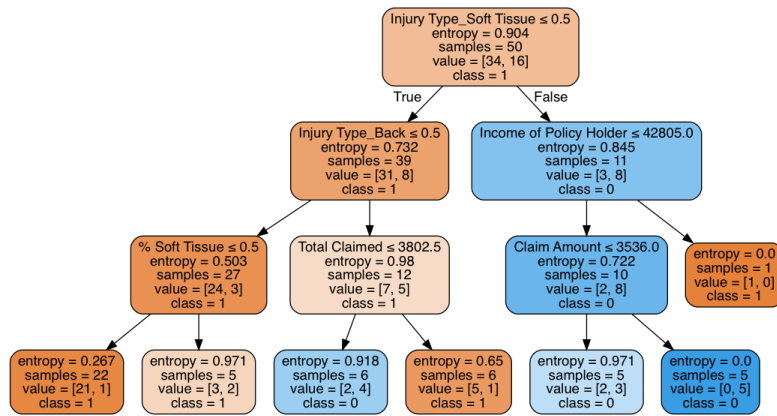
Accuracy: 0.644



This is not an approach good enough. The 10% - 90% train test split makes it difficult for the model to learn patterns from the limited training set.

3) Create a decision tree on the same data with max depth of 3 and an entropy measure. Does the accuracy change? Does the tree change? Discuss which measure you think is better.

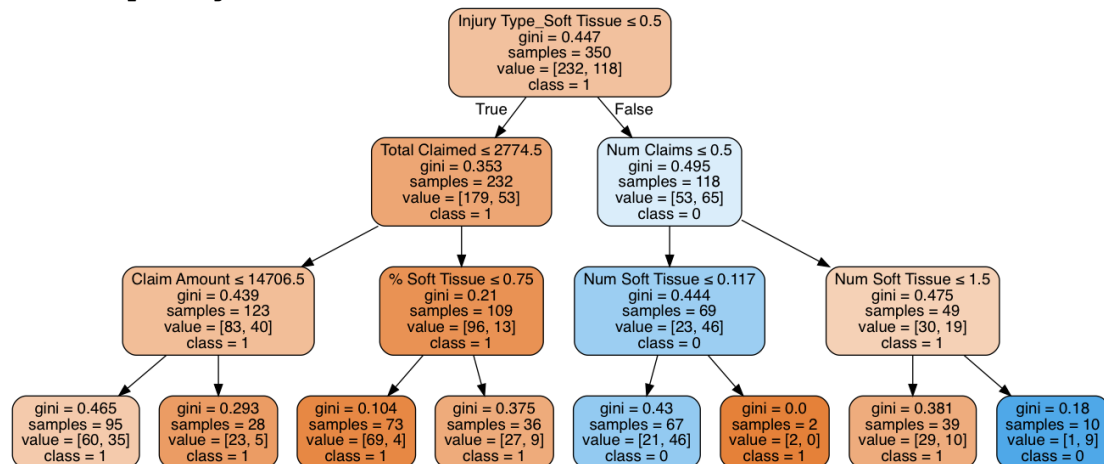
Accuracy: 0.644



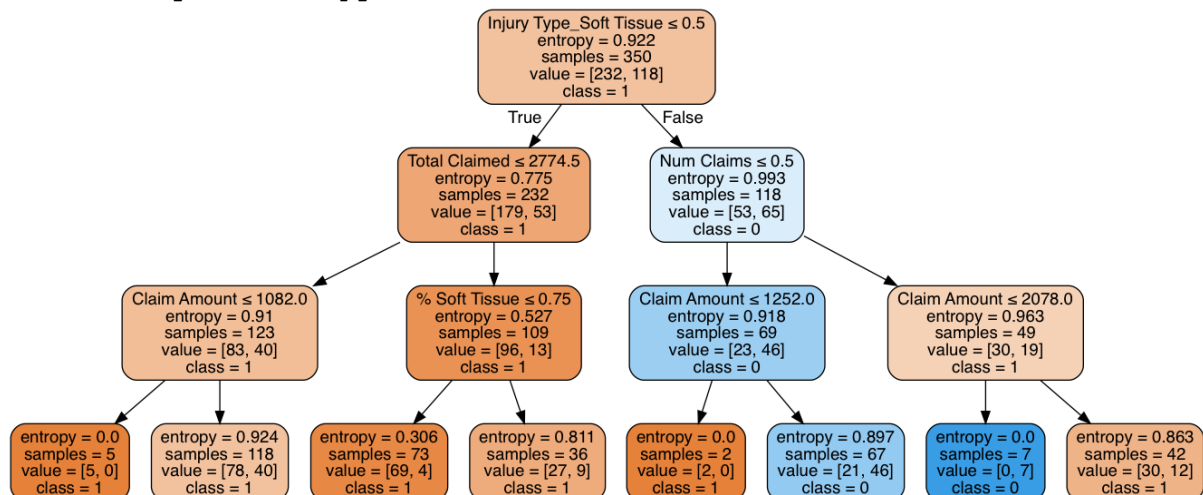
The accuracy doesn't change, but the tree does change a little bit. The left subtree under condition `Injury Type_Back ≤ 0.5` changes from feature "Num Soft Tissue" to "% Soft Tissue". For this case, the two trees share a same splitting performance so I don't tend to have any preference.

4) Now split the data into 70% train and 30% test using `random_state = 1`. Redo 2 and 3. Have the trees and accuracy changed? Are the trees more or less similar now? Discuss which split you think is better and why.

Accuracy of gini criterion: 0.727



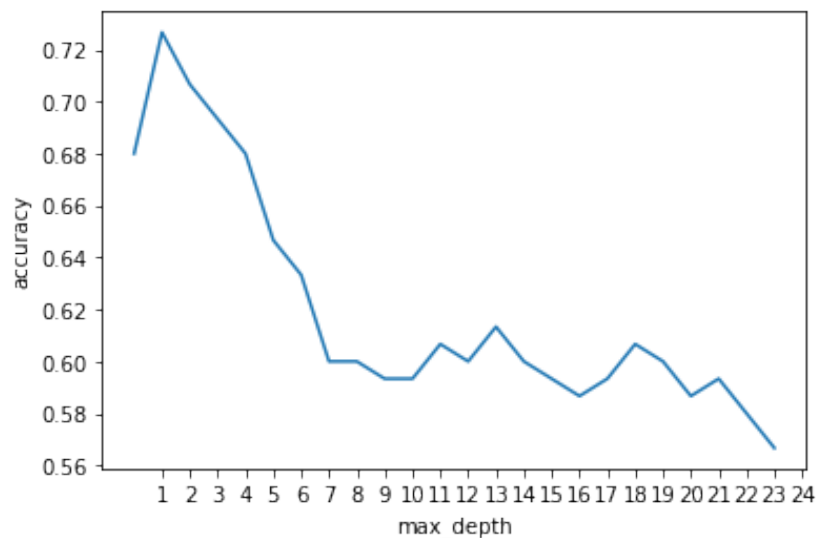
Accuracy of entropy criterion: 0.7



Both accuracy and trees change now. Gini outperformed entropy by .006. The trees are less similar now, and the difference is mainly on the splitting features and values on the second layer. Obviously the 70-30 split is better since more training data are fed into the model to learn the pattern.

5) Evaluate how the accuracy changes with the depth of the tree with the 70-30 data. Look at the accuracy for a max depth of 1, 2, 3, ... 10, 15, 20. Do you see underfitting? Do you see overfitting?

Below is the chart of accuracy changing with max_depth.



It could be observed that the performance peaks at max_depth of 2(.72), then gradually drops to .60 and oscillates up and down, displaying an overfitting that tends to split at deeper layers without improving accuracy. Although it “overfits”, I don’t think the model ever fits the data well with .72 accuracy, it could be further tuned to improve underfitting.

6) What variable provides the most information gain in the insurance fraud data (for the 70-30 split)?

In my case, it is the “Injury Type_Soft Tissue” after one-hot encoding the Injury Type column.

7) Decision trees are a “white box” method. What do you observe about the insurance fraud data using decision trees?

Decision trees apply a greedy methodology on every layer about choosing features. So we could directly observe which feature contributes most to the current subset. For the insurance data, the two criteria – gini and entropy tend to choose the same splits for the top 2 layers, and differ slightly on the third layer and beyond.

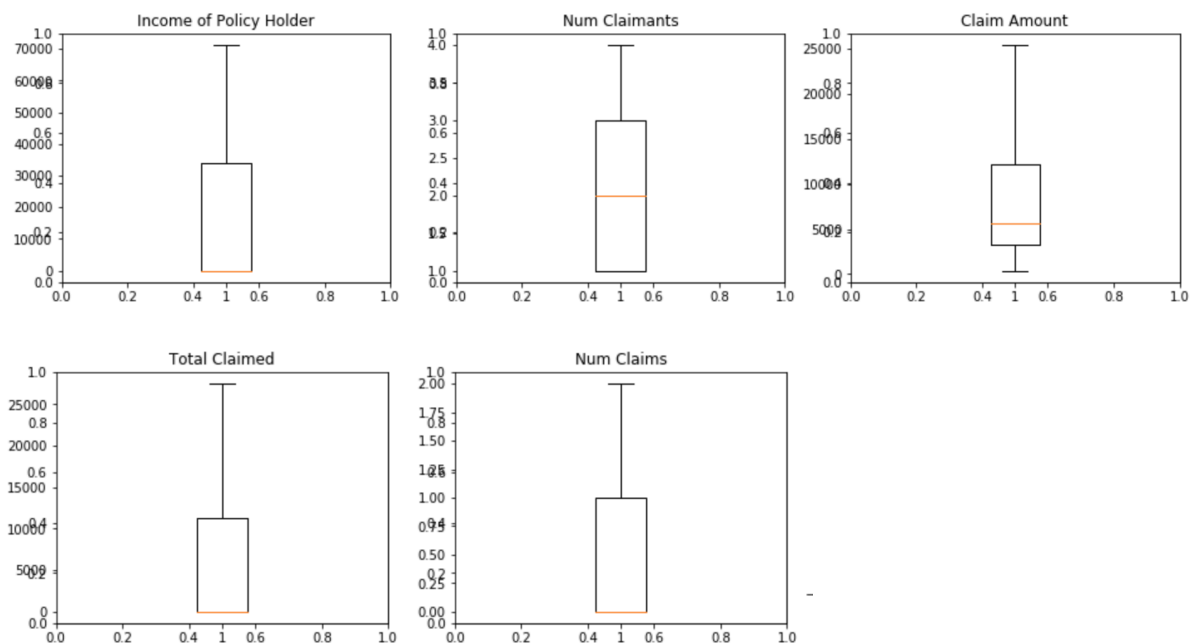
Also, a single decision tree would easily overfit by enabling deeper splits. So it’s a good idea to detect the point of overfitting thru test set performance, and use some bagging approaches such

as random forest to further improve the performance by a voting mechanism and a bootstrap sampling mechanism on both feature level and data level.

8) We talked about 'clamping' in class. Should we have done clamping on the data before using a decision tree?

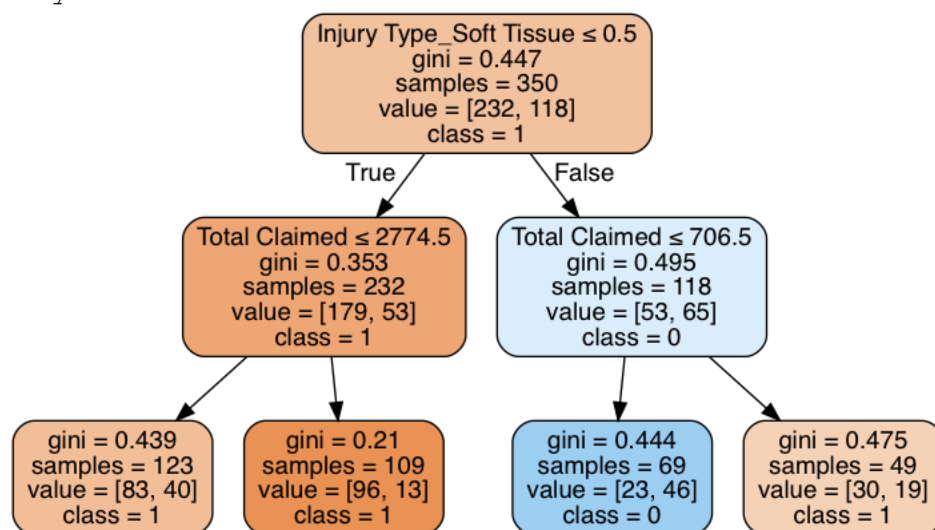
Theoretically, I don't think clamping would help improve accuracy in the decision tree case, since clamping reduces cutpoints in low density area, thus diminishing the number of potential splits towards these samples, though it may mitigate overfitting as well.

Experimentally, I tried clamping on the 5 numerical features based on boxplot:



Then I tried building decision trees containing the same hyperparameters as previous ones on this dataset. There was no dramatic change in performance, and the trees look similar as well.

Accuracy: 0.727



9) Read the paper Perner, 2013, on comparing decision trees. Does the proposed similarity measure find that your previous analysis is correct?

Let's try to calculate the similarity metric defined by the author on the previous decision trees.

For the 10-90 split data, the similarity between the gini tree and entropy tree is 1 since they're completely the same. While for the 70-30 split, the similarity between gini tree and entropy tree of 3 layers would be: $\text{avg}(0.9832 + 1 + 0.999055 + 0.66572) = 0.912$. In this way, the metric does reasonably quantify how similar two decision trees look like.

10) Are there problems with Perner, 2013, that you could improve upon? How about if there are different amounts of data in one tree? While Perner focuses on rules, what other measures could you use?

This similarity metric considers the Sim_{d_1, d_2} to be the average of every max Sim_{ij} for each rule. So a decision tree D_1 could achieve a very high similarity score with another tree D_2 if every single rule in D_1 has a close resemblance in D_2 . A special case is that, D_1 would get a high score if resembling a subtree in D_2 , while D_2 could contain far more other branches and leaves to differ from D_1 .

This could be achieved specially when D_2 is built on some more training data, whose patterns don't conflict with the original data set, so a new entire branch could be grown and differentiates it from D_1 .

In order to get a more precise measurement of tree similarity, I'm thinking of measuring it in a BFS way:

- 1) First assign an overall weight for every layer. All weights sum up 1 and the weight of a layer could correlate with its distance to the root, and the number of weights should equal to the number of layers for the deeper tree.
- 2) Inside each layer, Calculate the similarity of all the nodes for this layer between 2 trees, and the same method in Perner, 2013 could be applied here for both categorical and numerical nodes. Get the average of all node similarities, and multiply it with the layer weight as the layer similarity
- 3) Sum up all the layer similarity as the total similarity between two trees. In this way, if two trees differ a lot in depth, they tend to have a low similarity. And if a tree is a subtree of another, they will output a low score as well if the other tree contains far more other branches and leaves.